DevLog — Product Requirements Document

> **Your personal growth OS — built for developers who grow with intention, not by accident.**

Version: 1.0 · **Stack:** Next.js · Node.js/Express · PostgreSQL · Redis · BullMQ · Prisma · TypeScript

---

Table of Contents

1. Overview
2. Tech Stack
3. Project Structure
4. Database Schema
5. Module 1 — Authentication & Sessions
6. Module 2 — Dev Log (Learning Journal)
7. Module 3 — Goals & Milestones
8. Module 4 — Skill Tracker
9. Module 5 — Webhooks
10. Module 6 — Automated Workflows
11. RBAC — Role Based Access Control
12. Performance & Scalability
13. REST API Design Conventions
14. Frontend — Pages & UI
15. Build Phases & Implementation Order
16. Environment Variables

---

1. Overview

Problem

Developers grow by doing — but rarely by reflecting. You finish a project, learn something hard, fix a brutal bug, and move on. Six months later you cannot articulate what you have learned, where you are headed, or how far you have come.

There is no tool built specifically for a developer's growth. Notion is too freeform. LinkedIn is performative. GitHub shows *what* you built, not *who you became*.

Solution

DevLog is a private, self-hosted developer growth OS where you:

- **Log** what you learned every day in a structured format
- **Track** skills you are building and see measurable progress over time
- **Set** professional goals and break them into milestones
- **Reflect** through structured weekly prompts
- **Connect** your GitHub activity so commits automatically create log entries
- **Share** a read-only view with a mentor or future employer

What this project teaches (by design)

Concept	Where it lives
Next.js App Router, SSR, RSC	`apps/web/app/`
REST API design — versioned, proper HTTP semantics	`apps/api/src/modules/*/routes.ts`
JWT — access token, refresh token rotation, reuse detection	`apps/api/src/modules/auth/`
Server-side sessions — Redis-backed, encrypted	`apps/api/src/modules/auth/session.ts`
RBAC — role middleware, permission matrix	`apps/api/src/rbac/`
Webhooks — inbound (GitHub), outbound (notify), HMAC verify	`apps/api/src/modules/webhooks/`
Automated workflows — cron, rule engine, event triggers	`apps/api/src/modules/workflows/`
Async/await — every DB call, job dispatch, external API call	Throughout services
Cursor-based pagination	`apps/api/src/modules/logs/logs.repository.ts`
Redis caching — cache-aside pattern, TTL, invalidation	`apps/api/src/shared/cache.ts`
Job queue — BullMQ, worker, retry, dead-letter	`apps/api/src/shared/queue.ts`
Modular architecture — feature folders, clean service layer	`apps/api/src/modules/`

2. Tech Stack

```
apps/
  web/     → Next.js 14 (App Router, TypeScript, Tailwind)
  api/     → Node.js + Express (TypeScript, strict mode)

packages/
  db/      → Prisma schema, migrations, seed
  config/  → Typed env config shared across apps
```

Layer	Technology	Why
Frontend	Next.js 14 — App Router	SSR, RSC, file-based routing, streaming
Backend	Node.js + Express	Explicit middleware, teaches the full HTTP lifecycle
Database	PostgreSQL + Prisma	Type-safe queries, migration history
Cache	Redis	Skill graph cache, session store, rate-limit counters
Job Queue	BullMQ (Redis)	Async webhook dispatch, notification jobs
Auth	JWT (RS256) + HttpOnly cookie	Manual implementation — no Passport.js
Monorepo	Turborepo	Build order: db → api → web
Language	TypeScript (strict)	Shared types across frontend and backend

---

3. Project Structure

```
devlog/
├── apps/
│   ├── web/                        # Next.js frontend
│   │   ├── app/
│   │   │   ├── layout.tsx          # Root layout — RSC, font, providers
│   │   │   ├── (auth)/
│   │   │   │   ├── login/page.tsx  # Login page
│   │   │   │   └── register/page.tsx
│   │   │   └── (app)/              # Protected routes
│   │   │       ├── dashboard/page.tsx
│   │   │       ├── logs/
│   │   │       │   ├── page.tsx    # Log feed — infinite scroll
│   │   │       │   └── [id]/page.tsx # Single log entry
```

```
│   │   │       ├── goals/
│   │   │       │   ├── page.tsx
│   │   │       │   └── [id]/page.tsx
│   │   │       ├── skills/page.tsx
│   │   │       ├── reflections/page.tsx
│   │   │       ├── settings/
│   │   │       │   ├── page.tsx
│   │   │       │   └── webhooks/page.tsx
│   │   │       └── mentor/[token]/page.tsx  # Public read-only share
│   │   ├── components/
│   │   │   ├── LogCard.tsx
│   │   │   ├── SkillBadge.tsx
│   │   │   ├── GoalProgress.tsx
│   │   │   ├── RoleGate.tsx          # RBAC client guard
│   │   │   ├── AuthProvider.tsx      # Token refresh + context
│   │   │   └── ui/                   # Reusable primitives
│   │   └── lib/
│   │       ├── api.ts                # Typed fetch wrapper + 401 handler
│   │       └── session.ts            # Server-side session reader
│   │
│   └── api/                          # Express backend
│       └── src/
│           ├── app.ts                # Express app — middleware, routes
│           ├── server.ts             # HTTP server — startup, shutdown
│           ├── modules/
│           │   ├── auth/
│           │   │   ├── auth.routes.ts
│           │   │   ├── auth.controller.ts
│           │   │   ├── auth.service.ts      # JWT core
│           │   │   ├── auth.middleware.ts   # verifyAccessToken
│           │   │   └── session.ts           # Redis session manager
│           │   ├── logs/
│           │   │   ├── logs.routes.ts
│           │   │   ├── logs.controller.ts
│           │   │   ├── logs.service.ts
│           │   │   └── logs.repository.ts   # Cursor pagination
│           │   ├── goals/
│           │   │   ├── goals.routes.ts
│           │   │   ├── goals.controller.ts
│           │   │   ├── goals.service.ts
│           │   │   └── goals.repository.ts
│           │   ├── skills/
│           │   │   ├── skills.routes.ts
│           │   │   ├── skills.controller.ts
│           │   │   ├── skills.service.ts
│           │   │   └── skills.repository.ts
│           │   ├── reflections/
```

```
│           │   │   ├── reflections.routes.ts
│           │   │   ├── reflections.controller.ts
│           │   │   └── reflections.service.ts
│           │   ├── webhooks/
│           │   │   ├── webhook.routes.ts
│           │   │   ├── webhook.service.ts      # Inbound — verify + process
│           │   │   └── webhook.dispatcher.ts   # Outbound — retry + backoff
│           │   └── workflows/
│           │       ├── workflow.engine.ts      # Rule evaluator
│           │       └── workflow.cron.ts        # Scheduled jobs
│           ├── rbac/
│           │   ├── rbac.middleware.ts          # requireRole()
│           │   └── permissions.ts             # Permission matrix
│           └── shared/
│               ├── cache.ts                   # Redis get/set/invalidate
│               ├── queue.ts                   # BullMQ producer/consumer
│               ├── logger.ts                  # Pino structured logger
│               └── errors.ts                  # AppError hierarchy
│
└── packages/
    ├── db/
    │   ├── schema.prisma
    │   ├── migrations/
    │   └── seed.ts
    └── config/
        └── index.ts                           # Zod-validated env config
```

---

4. Database Schema

```prisma
```

```
// packages/db/schema.prisma

model User {
  id            String    @id @default(cuid())
  email         String    @unique
  passwordHash  String
  name          String
  role          Role      @default(MEMBER)
  shareToken    String?   @unique   // read-only mentor share link
  createdAt     DateTime  @default(now())

  logs          Log[]
  goals         Goal[]
  skills        UserSkill[]
  reflections   Reflection[]
  refreshTokens RefreshToken[]
  webhooks      Webhook[]
  workflows     WorkflowRule[]
}

model RefreshToken {
  id        String   @id @default(cuid())
  tokenHash String   @unique          // SHA-256 of the raw token
  userId    String
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade
  deviceId  String                    // browser fingerprint or device label
  expiresAt DateTime
  usedAt    DateTime?                 // detect reuse — if set, token already rotate
  createdAt DateTime @default(now())

  @@index([userId])
}

model Log {
  id          String    @id @default(cuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id], onDelete: Casc
  title       String
  body        String                  // markdown content
  mood        Mood      @default(NEUTRAL)
  tags        String[]
  source      LogSource @default(MANUAL)  // MANUAL | GITHUB_PUSH | GITHUB_PR
  githubRef   String?                // commit SHA or PR number if source = GITHUB_*
  skills      LogSkill[]             // skills referenced in this log
  goals       LogGoal[]              // goals this log contributes to
  createdAt   DateTime  @default(now())
```

```
  updatedAt   DateTime  @updatedAt

  @@index([userId, createdAt(sort: Desc)])
  @@index([userId, tags])
}

model Goal {
  id          String      @id @default(cuid())
  userId      String
  user        User        @relation(fields: [userId], references: [id], onDelete: Ca
  title       String
  description String?
  status      GoalStatus  @default(IN_PROGRESS)
  targetDate  DateTime?
  milestones  Milestone[]
  logs        LogGoal[]
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt

  @@index([userId, status])
}

model Milestone {
  id          String    @id @default(cuid())
  goalId      String
  goal        Goal      @relation(fields: [goalId], references: [id], onDelete: Casc
  title       String
  completed   Boolean   @default(false)
  completedAt DateTime?
  order       Int
}

model Skill {
  id    String      @id @default(cuid())
  name  String      @unique   // "TypeScript", "System Design", "PostgreSQL"
  category SkillCategory
  users UserSkill[]
  logs  LogSkill[]
}

model UserSkill {
  id        String   @id @default(cuid())
  userId    String
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade
  skillId   String
  skill     Skill    @relation(fields: [skillId], references: [id])
  level     Int      @default(1)   // 1-10
```

```
  logCount  Int      @default(0)   // auto-incremented when a log tags this skill
  updatedAt DateTime @updatedAt

  @@unique([userId, skillId])
  @@index([userId])
}

model LogSkill {
  logId   String
  log     Log    @relation(fields: [logId], references: [id], onDelete: Cascade)
  skillId String
  skill   Skill  @relation(fields: [skillId], references: [id])
  @@id([logId, skillId])
}

model LogGoal {
  logId  String
  log    Log    @relation(fields: [logId], references: [id], onDelete: Cascade)
  goalId String
  goal   Goal   @relation(fields: [goalId], references: [id])
  @@id([logId, goalId])
}

model Reflection {
  id        String   @id @default(cuid())
  userId    String
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade
  weekStart DateTime               // Monday 00:00:00 UTC
  wins      String                 // markdown
  struggles String                 // markdown
  learning  String                 // markdown
  nextWeek  String                 // markdown
  mood      Mood
  createdAt DateTime @default(now())

  @@unique([userId, weekStart])
  @@index([userId])
}

model Webhook {
  id          String        @id @default(cuid())
  userId      String
  user        User          @relation(fields: [userId], references: [id], onDelete:
  type        WebhookType                          // INBOUND | OUTBOUND
  source      String?                              // "github" for inbound
  targetUrl   String?                              // for outbound
  secret      String                               // HMAC secret (stored encrypted)
```

```
  events      String[]                          // ["goal.completed", "streak.brok
  active      Boolean       @default(true)
  lastTriggeredAt DateTime?
  createdAt   DateTime      @default(now())
}

model WorkflowRule {
  id        String   @id @default(cuid())
  userId    String
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade
  name      String
  trigger   String                // "goal.completed" | "streak.3" | "weekly.reflect
  action    String                // "send.webhook" | "send.email" | "create.badge"
  config    Json                   // action-specific config
  active    Boolean  @default(true)
  createdAt DateTime @default(now())
}

enum Role          { OWNER ADMIN MEMBER VIEWER }
enum Mood          { GREAT GOOD NEUTRAL TIRED FRUSTRATED }
enum GoalStatus    { IN_PROGRESS COMPLETED PAUSED ABANDONED }
enum LogSource     { MANUAL GITHUB_PUSH GITHUB_PR }
enum WebhookType   { INBOUND OUTBOUND }
enum SkillCategory { LANGUAGE FRAMEWORK DATABASE DEVOPS SOFTSKILL CONCEPT }
```

---

5. Module 1 — Authentication & Sessions

What it teaches

- JWT signing with RS256 (asymmetric keys)
- Access token in memory, refresh token in HttpOnly cookie
- Refresh token rotation and reuse detection
- Server-side Redis sessions for admin flows
- Express middleware chains — each concern in its own function

JWT Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│  Login flow                                                          │
│                                                                      │
│  POST /auth/login                                                    │
│    → verify password (bcrypt compare)                                │
```

```
|    → generate access token  (RS256, 15 min, userId+role payload) |
|    → generate refresh token (random 32 bytes, store hash in DB)  |
|    → set refresh token as HttpOnly cookie (SameSite=Strict)      |
|    → return { accessToken, user } in JSON body                   |
└──────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────┐
|  Refresh flow                                                    |
|                                                                  |
|  POST /auth/refresh                                              |
|    → read refresh token from cookie                              |
|    → hash it and look up in DB                                   |
|    → if token.usedAt is set → REUSE DETECTED → revoke all tokens |
|    → mark old token as used (token.usedAt = now)                 |
|    → issue new access token + new refresh token                  |
|    → store new refresh token hash, set new cookie                |
└──────────────────────────────────────────────────────────────────┘
```

Why access token lives in memory (not localStorage)

`localStorage` is accessible to any JavaScript running on the page, making it vulnerable to XSS. The access token is stored in a JavaScript variable inside `AuthProvider.tsx`. When the page refreshes, `AuthProvider` fires a silent `/auth/refresh` call on mount using the HttpOnly cookie (which JavaScript cannot read) to get a new access token.

API Endpoints

Method	Route	Body / Params	Response
`POST`	`/api/v1/auth/register`	`{ email, password, name }`	`{ accessToken, user }` + sets cookie
`POST`	`/api/v1/auth/login`	`{ email, password }`	`{ accessToken, user }` + sets cookie
`POST`	`/api/v1/auth/refresh`	— (reads cookie)	`{ accessToken }` + rotates cookie
`POST`	`/api/v1/auth/logout`	—	Clears cookie, deletes token from DB
`POST`	`/api/v1/auth/logout-all`	—	Deletes ALL refresh tokens for user
`GET`	`/api/v1/auth/me`	—	`{ user }`

Middleware chain (applied to every protected route)

```typescript
// apps/api/src/modules/auth/auth.middleware.ts

export const verifyAccessToken = (req, res, next) => {
  // 1. Extract Bearer token from Authorization header
  // 2. Verify signature using public key (RS256)
  // 3. Check expiry
  // 4. Attach decoded payload to req.tokenPayload
  // 5. Call next() or throw 401
}

export const attachUser = async (req, res, next) => {
  // 1. Read userId from req.tokenPayload
  // 2. Load user from DB (cached in Redis for 5 min)
  // 3. Attach to req.user
  // 4. Call next() or throw 401 if user deleted/suspended
}

// Usage in routes:
router.get('/logs', verifyAccessToken, attachUser, requireRole('MEMBER'), logsContro
```

Session management (admin flows)

```typescript
// apps/api/src/modules/auth/session.ts

// Sessions are stored in Redis with key: session:{sessionId}
// Session data is AES-256 encrypted before storing
// TTL: 2 hours, sliding window (reset on each request)
// Used only for admin dashboard routes, not the main app API

export const createSession = async (userId: string, role: string): Promise<string>
export const getSession = async (sessionId: string): Promise<Session | null>
export const destroySession = async (sessionId: string): Promise<void>
```

---

6. Module 2 — Dev Log (Learning Journal)

What it teaches

- Full REST API design: proper HTTP verbs, status codes, resource naming

- Cursor-based pagination (why offset pagination fails at scale)
- Repository pattern — DB logic isolated from business logic
- Async/await throughout — every DB call is explicitly awaited
- Full-text search with PostgreSQL `tsvector`

Data model

A `Log` is a markdown journal entry about something you learned or did. It has a mood, tags, optional skill links, and optional goal links. Logs can be created manually or auto-created by an inbound GitHub webhook.

API Endpoints

Method	Route	Description
`GET`	`/api/v1/logs`	List logs — cursor paginated, filterable by tag/skill/date range
`POST`	`/api/v1/logs`	Create a new log entry
`GET`	`/api/v1/logs/:id`	Get a single log entry
`PATCH`	`/api/v1/logs/:id`	Update a log entry
`DELETE`	`/api/v1/logs/:id`	Delete a log entry
`GET`	`/api/v1/logs/search?q=`	Full-text search across title and body

Cursor-based pagination

```
GET /api/v1/logs?limit=20&cursor=clxyz123

Why cursor, not offset:
  → offset=100 means "skip 100 rows" — the DB still reads and discards 100 rows
  → cursor means "give me rows after this specific record" — O(1) with an index
  → offset breaks when new items are inserted (page 2 skips or repeats items)
  → cursor is stable — the same cursor always returns the same next page

Response shape:
{
  data: Log[],
  nextCursor: "clxyz456" | null,   ← null means no more pages
  hasMore: boolean
}
```

```
Implementation:
  SELECT * FROM logs
  WHERE userId = $1
    AND createdAt < (SELECT createdAt FROM logs WHERE id = $cursor)
  ORDER BY createdAt DESC
  LIMIT $limit + 1          ← fetch one extra to determine hasMore
```

Controller → Service → Repository pattern

```
Request
  → logs.controller.ts    validate input (zod), call service, return response
      → logs.service.ts   business logic — check ownership, apply skill side
effects
          → logs.repository.ts  all SQL/Prisma queries live here, nothing else
```

Why this separation? The controller never touches the database. The repository never contains business logic. This means you can swap PostgreSQL for another database by rewriting only the repository. You can test the service by mocking the repository.

Side effects when a log is saved

1. For each skill tag in the log, `UserSkill.logCount` is incremented (in the same DB transaction)
2. For each goal linked to the log, check if any milestone should be auto-marked complete
3. Publish a `log.created` event to the workflow engine so automation rules can fire
4. Invalidate the skill graph Redis cache for this user

All four steps happen inside `logs.service.ts` using a Prisma transaction to guarantee atomicity.

---

7. Module 3 — Goals & Milestones

What it teaches

- Nested resources in REST (`/goals/:id/milestones`)
- Transactions — goal status update must be atomic with milestone completion
- Computed fields — progress percentage derived from milestones, not stored
- Triggering events from service layer (fires workflow rules)

API Endpoints

Method	Route	Description
`GET`	`/api/v1/goals`	List goals, filterable by status
`POST`	`/api/v1/goals`	Create a goal
`GET`	`/api/v1/goals/:id`	Get goal with milestones and linked logs
`PATCH`	`/api/v1/goals/:id`	Update goal title, description, targetDate, status
`DELETE`	`/api/v1/goals/:id`	Delete goal and all its milestones
`POST`	`/api/v1/goals/:id/milestones`	Add a milestone
`PATCH`	`/api/v1/goals/:id/milestones/:mid`	Mark milestone complete / reorder
`DELETE`	`/api/v1/goals/:id/milestones/:mid`	Delete milestone

Goal completion logic

typescript

```typescript
// apps/api/src/modules/goals/goals.service.ts

async completeMilestone(goalId: string, milestoneId: string, userId: string) {
  return await prisma.$transaction(async (tx) => {
    // 1. Mark milestone as completed
    await tx.milestone.update({ where: { id: milestoneId }, data: { completed: true,

    // 2. Check if ALL milestones for this goal are now complete
    const allMilestones = await tx.milestone.findMany({ where: { goalId } })
    const allDone = allMilestones.every(m => m.completed)

    // 3. If all done, mark goal as COMPLETED
    if (allDone) {
      await tx.goal.update({ where: { id: goalId }, data: { status: 'COMPLETED' } })
      // 4. Emit event — workflow engine picks this up to fire automations
      await workflowEngine.emit('goal.completed', { goalId, userId })
    }
  })
}
```

8. Module 4 — Skill Tracker

What it teaches

- Derived/computed data — skill level calculated from log activity, not manually set
- Redis cache-aside pattern — cache the skill graph, invalidate on write
- Aggregation queries — how to count related records efficiently in SQL

Skill level algorithm

```
Level is computed from logCount (number of logs that tagged this skill):

  1-2 logs    → level 1  (Aware)
  3-5 logs    → level 2  (Exploring)
  6-10 logs   → level 3  (Practising)
  11-20 logs  → level 4  (Comfortable)
  21-35 logs  → level 5  (Proficient)
  36-55 logs  → level 6  (Strong)
  56-80 logs  → level 7  (Advanced)
  81-110 logs → level 8  (Expert)
  111-149     → level 9  (Master)
  150+        → level 10 (Distinguished)

This makes levels feel earned — you cannot manually bump your level.
```

API Endpoints

Method	Route	Description
`GET`	`/api/v1/skills`	List all skills (global catalog)
`GET`	`/api/v1/skills/me`	Current user's skill levels — cached in Redis
`GET`	`/api/v1/skills/me/graph`	Skill graph data for radar chart — cached
`POST`	`/api/v1/skills`	Add a skill to the global catalog (admin only)

Redis caching (cache-aside pattern)

```typescript
```

```
// apps/api/src/shared/cache.ts

export const cacheAside = async <T>(
  key: string,
  ttlSeconds: number,
  fetchFn: () => Promise<T>
): Promise<T> => {
  // 1. Try to read from Redis
  const cached = await redis.get(key)
  if (cached) return JSON.parse(cached)

  // 2. Cache miss — fetch from DB
  const data = await fetchFn()

  // 3. Store in Redis with TTL
  await redis.setex(key, ttlSeconds, JSON.stringify(data))
  return data
}

// Usage in skills.service.ts:
const skillGraph = await cacheAside(
  `skill-graph:${userId}`,
  300,  // 5 minute TTL
  () => skillsRepository.getSkillGraph(userId)
)

// Invalidation — called whenever a log is saved or deleted:
await redis.del(`skill-graph:${userId}`)
```

---

9. Module 5 — Webhooks

What it teaches

- **Where** a webhook receiver lives in a REST API
- HMAC-SHA256 signature verification — how to prove a request came from GitHub
- Idempotency — same event must never create duplicate records
- **Where** to trigger outbound webhooks — after a DB write commits, never before
- Retry logic with exponential backoff using BullMQ

9.1 Inbound Webhooks (GitHub → DevLog)

When you push a commit to a connected GitHub repo, GitHub sends a POST request to your DevLog instance. DevLog automatically creates a log entry from the commit message.

```
GitHub push event
  → POST /api/v1/webhooks/github
      → webhook.service.ts
          1. Read X-Hub-Signature-256 header
          2. Compute HMAC-SHA256(secret, rawBody)
          3. Compare using timingSafeEqual (prevents timing attacks)
          4. If invalid → 401 Unauthorized, log attempt
          5. Parse event type from X-GitHub-Event header
          6. Check idempotency: has this delivery ID been processed?
             (store github delivery ID in Redis with TTL 24h)
          7. If already processed → 200 OK (do nothing, acknowledge)
          8. If new → create Log record from commit data
          9. Mark delivery ID as processed in Redis
```

Why check idempotency? GitHub retries webhook deliveries if your server returns a non-2xx response, or if it doesn't hear back within 10 seconds. Without idempotency checking, a slow server could process the same commit push twice and create duplicate log entries.

Signature verification code

```typescript
// apps/api/src/modules/webhooks/webhook.service.ts

import { createHmac, timingSafeEqual } from 'crypto'

const verifyGithubSignature = (secret: string, rawBody: Buffer, signature: string):
  const expected = 'sha256=' + createHmac('sha256', secret).update(rawBody).digest('
  // timingSafeEqual prevents timing attacks — always compare same-length buffers
  return timingSafeEqual(Buffer.from(expected), Buffer.from(signature))
}
```

9.2 Outbound Webhooks (DevLog → Your services)

Users can register a URL to receive notifications when things happen in DevLog (goal completed, streak broken, weekly reflection due).

```
Trigger point (IMPORTANT):
  Outbound webhooks are dispatched AFTER the DB transaction commits.
  Never dispatch inside a transaction — if the transaction rolls back,
  the webhook has already been sent and cannot be recalled.

Dispatch flow:
  1. DB write commits (e.g. goal status → COMPLETED)
  2. goals.service.ts calls webhookDispatcher.dispatch('goal.completed', payload)
  3. dispatcher pushes a job to the BullMQ 'webhook-dispatch' queue
     (this is instant and non-blocking — the API response is not delayed)
  4. BullMQ worker picks up the job asynchronously
  5. Worker POSTs to the user's registered URL with HMAC signature
  6. If response is not 2xx:
       - Retry after 30s, then 2min, then 10min, then 1h, then 6h (5 attempts)
       - If all attempts fail → move to dead-letter queue, notify user
  7. Update webhook.lastTriggeredAt in DB
```

API Endpoints

Method	Route	Description
`GET`	`/api/v1/webhooks`	List user's registered webhooks
`POST`	`/api/v1/webhooks`	Register a new outbound webhook URL
`DELETE`	`/api/v1/webhooks/:id`	Remove a webhook
`GET`	`/api/v1/webhooks/:id/deliveries`	View delivery history for a webhook
`POST`	`/api/v1/webhooks/github/setup`	Connect GitHub repo (returns inbound URL + secret)

---

10. Module 6 — Automated Workflows

What it teaches

- When to use cron vs event triggers
- Rule engine pattern — decouple trigger detection from action execution
- Event bus — services publish events, rules subscribe to them
- Where automation fits in the request lifecycle (always async, never blocking)

Workflow rules

A `WorkflowRule` is a user-configured rule with a trigger and an action.

```
trigger: "goal.completed"    action: "send.webhook"   → notify Slack when a goal
is done
trigger: "streak.broken"     action: "send.email"     → email yourself when you
miss 3 days
trigger: "streak.7"          action: "create.badge"   → earn a 7-day streak badge
trigger: "weekly.reflection" action: "send.email"     → remind yourself every
Monday morning
```

Rule engine

```typescript
// apps/api/src/modules/workflows/workflow.engine.ts

class WorkflowEngine {
  // Publish an event — called from service layer after DB commits
  async emit(trigger: string, context: Record<string, unknown>): Promise<void> {
    // 1. Find all active WorkflowRules where trigger matches
    const rules = await prisma.workflowRule.findMany({
      where: { trigger, active: true, userId: context.userId as string }
    })

    // 2. For each matching rule, push an action job to BullMQ
    //    (async — does not block the caller)
    for (const rule of rules) {
      await actionQueue.add('execute-action', { rule, context })
    }
  }
}

// BullMQ worker processes each action job:
actionWorker.process('execute-action', async (job) => {
  const { rule, context } = job.data
  switch (rule.action) {
    case 'send.webhook':  return executeWebhookAction(rule.config, context)
    case 'send.email':    return executeEmailAction(rule.config, context)
    case 'create.badge':  return executeBadgeAction(rule.config, context)
  }
})
```

Cron jobs

```typescript
// apps/api/src/modules/workflows/workflow.cron.ts

// Runs every Monday at 08:00 in user's timezone
cron.schedule('0 8 * * 1', async () => {
  // 1. Find all users who have not submitted a reflection for this week
  // 2. For each such user, fire 'weekly.reflection' trigger
  // 3. WorkflowEngine picks it up — if user has a rule for it, action executes
})

// Runs every day at 00:01
cron.schedule('1 0 * * *', async () => {
  // 1. Find all users whose last log was more than 2 days ago
  // 2. Calculate streak (consecutive days with at least one log)
  // 3. If streak broken → emit 'streak.broken' event
  // 4. If streak hit milestone (3, 7, 14, 30, 100) → emit 'streak.N' event
})
```

Cron vs event trigger — when to use which:

Use cron when...	Use event trigger when...
The thing happens on a schedule regardless of user action	The thing is a direct consequence of a user action
Weekly reflection reminder	Goal completed
Daily streak check	Log created
Monthly progress report generation	Webhook delivery failed

---

11. RBAC — Role Based Access Control

What it teaches

- Middleware-based RBAC — roles enforced at the route level
- Permission matrix — all access rules in one place, not scattered as if-checks
- Resource ownership — a MEMBER can edit their own logs but not another user's

Roles

Role	Who has it	What they can do
`OWNER`	The person who deployed the instance	Everything. Manage all users, all data, system config.
`ADMIN`	Trusted users elevated by OWNER	Manage users, view all data, cannot change system config.
`MEMBER`	Default role for all registered users	Full access to their own data only.
`VIEWER`	Mentor or read-only guest	Read-only access to a specific user's shared data via share token.

Permission matrix

```typescript
// apps/api/src/rbac/permissions.ts

export const permissions = {
  'logs:create':          ['OWNER', 'ADMIN', 'MEMBER'],
  'logs:read:own':        ['OWNER', 'ADMIN', 'MEMBER'],
  'logs:read:any':        ['OWNER', 'ADMIN'],
  'logs:update:own':      ['OWNER', 'ADMIN', 'MEMBER'],
  'logs:update:any':      ['OWNER', 'ADMIN'],
  'logs:delete:own':      ['OWNER', 'ADMIN', 'MEMBER'],
  'logs:delete:any':      ['OWNER'],
  'goals:create':         ['OWNER', 'ADMIN', 'MEMBER'],
  'goals:read:own':       ['OWNER', 'ADMIN', 'MEMBER'],
  'users:list':           ['OWNER', 'ADMIN'],
  'users:delete':         ['OWNER'],
  'webhooks:manage':      ['OWNER', 'ADMIN', 'MEMBER'],
  'system:config':        ['OWNER'],
} as const

export type Permission = keyof typeof permissions
```

RBAC middleware

```typescript
```

```
// apps/api/src/rbac/rbac.middleware.ts

export const requireRole = (permission: Permission) => {
  return (req: Request, res: Response, next: NextFunction) => {
    const userRole = req.user.role

    // Check if the user's role is in the permission's allowed roles array
    if (!permissions[permission].includes(userRole)) {
      throw new ForbiddenError(`Role ${userRole} cannot perform ${permission}`)
    }

    // Resource ownership check for :own permissions
    if (permission.endsWith(':own') && req.params.userId) {
      if (req.params.userId !== req.user.id) {
        throw new ForbiddenError('You can only access your own resources')
      }
    }

    next()
  }
}

// Usage in route definitions:
router.get('/logs',        verifyAccessToken, attachUser, requireRole('logs:read:own
router.get('/admin/users', verifyAccessToken, attachUser, requireRole('users:list'),
router.delete('/users/:id', verifyAccessToken, attachUser, requireRole('users:delete
```

---

12. Performance & Scalability

What it teaches

- Where and why to add indexes
- Cache-aside pattern and cache invalidation
- Async job queues — offloading work from the request thread
- Graceful shutdown — why it matters and how to implement it
- Multi-user ready architecture from day one

Database indexes

```sql
```

```sql
-- Every query that filters by userId gets an index
-- Every query that sorts by createdAt gets a composite index

-- logs table — the most-queried table
CREATE INDEX idx_logs_user_created   ON logs(user_id, created_at DESC);
CREATE INDEX idx_logs_user_tags      ON logs USING GIN(tags);          -- array sear
CREATE INDEX idx_logs_fulltext       ON logs USING GIN(to_tsvector('english', title

-- goals table
CREATE INDEX idx_goals_user_status   ON goals(user_id, status);

-- refresh_tokens table — looked up on every API request
CREATE UNIQUE INDEX idx_refresh_token_hash ON refresh_tokens(token_hash);
CREATE INDEX idx_refresh_tokens_user       ON refresh_tokens(user_id);
```

What is cached in Redis and why

Cache key	TTL	Invalidated when
`skill-graph:{userId}`	5 min	Log created/deleted
`user:{userId}`	5 min	User profile updated
`skills:catalog`	1 hour	New skill added to catalog
`streak:{userId}`	1 min	Log created (cron recomputes)

BullMQ job queues

All heavy or non-critical work is offloaded to queues. The API response returns immediately; the work happens in the background.

```
'webhook-dispatch'  → outbound webhook delivery with retry
'send-email'        → transactional emails (weekly reflection prompt, streak
alert)
'compute-streak'    → streak recalculation after a log is created
'generate-report'   → monthly progress report PDF generation
```

Graceful shutdown

```typescript
```

```
// apps/api/src/server.ts

process.on('SIGTERM', async () => {
  // 1. Stop accepting new HTTP connections
  server.close()
  // 2. Wait for in-flight requests to finish (max 30s)
  // 3. Close BullMQ workers (wait for current jobs to complete)
  await worker.close()
  // 4. Close DB connection pool
  await prisma.$disconnect()
  // 5. Close Redis connection
  await redis.quit()
  process.exit(0)
})
```

Scalability foundation decisions

Decision	Why
Feature-based module folders	Adding a new feature never touches existing modules
Repository pattern	Swap database without changing business logic
BullMQ for all async work	Horizontal scale: run more workers without touching the API
Stateless JWT auth	Run multiple API instances — no shared session state needed
Redis for sessions and cache	Redis is already in the stack, centralised state is safe to share
Prisma migrations	Schema changes are versioned, reproducible, never ad-hoc
Zod for all input validation	All external input is validated and typed before touching business logic

---

13. REST API Design Conventions

URL structure

```
/api/v1/{resource}                    collection
/api/v1/{resource}/{id}               single resource
/api/v1/{resource}/{id}/{sub}         sub-collection
/api/v1/{resource}/{id}/{sub}/{sid}   single sub-resource
```

HTTP verbs and what they mean

Verb	Meaning	Body	Idempotent
`GET`	Read — never mutates state	—	Yes
`POST`	Create a new resource	Full resource	No
`PATCH`	Partial update — only send changed fields	Partial	No
`PUT`	Full replace — send entire resource	Full resource	Yes
`DELETE`	Delete a resource	—	Yes

> **Rule:** If the operation creates something new, use `POST`. If it changes something that already exists, use `PATCH`. Never use `GET` to mutate state.

Status codes

Code	When to use
`200 OK`	Successful GET, PATCH, DELETE
`201 Created`	Successful POST that created a resource
`204 No Content`	Successful DELETE with no body
`400 Bad Request`	Validation error — missing or invalid fields
`401 Unauthorized`	No token, or token invalid/expired
`403 Forbidden`	Token valid, but role does not have permission
`404 Not Found`	Resource does not exist or user has no access
`409 Conflict`	Duplicate — e.g. email already registered
`422 Unprocessable Entity`	Request understood but semantically invalid
`429 Too Many Requests`	Rate limit exceeded
`500 Internal Server Error`	Unexpected server error — always log these

Response envelope

```typescript
```

```
// Success
{
  "data": { ... } | [ ... ],
  "meta": {                     // present on paginated list responses
    "nextCursor": "clxyz456",
    "hasMore": true,
    "total": 142                // optional — expensive to compute, only include if n
  }
}

// Error
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "email is required",
    "details": [                // optional — field-level errors from Zod
      { "field": "email", "message": "Required" }
    ]
  }
}
```

Input validation with Zod (every endpoint)

```typescript
// apps/api/src/modules/logs/logs.controller.ts

const createLogSchema = z.object({
  title:    z.string().min(1).max(200),
  body:     z.string().min(1),
  mood:     z.enum(['GREAT','GOOD','NEUTRAL','TIRED','FRUSTRATED']).default('NEUTRAL
  tags:     z.array(z.string()).max(10).default([]),
  skillIds: z.array(z.string().cuid()).max(10).default([]),
  goalIds:  z.array(z.string().cuid()).max(5).default([]),
})

export const create = async (req: Request, res: Response) => {
  const input = createLogSchema.parse(req.body)   // throws ZodError if invalid
  const log = await logsService.create(req.user.id, input)
  res.status(201).json({ data: log })
}
```

14. Frontend — Pages & UI

Layout structure

```
RootLayout (layout.tsx)
  ├── AuthProvider          ← holds access token in memory, runs silent refresh on
mount
  ├── (auth) group          ← unauthenticated pages, no sidebar
  │   ├── /login
  │   └── /register
  └── (app) group           ← authenticated pages, with sidebar
      ├── Sidebar
      ├── /dashboard
      ├── /logs
      ├── /goals
      ├── /skills
      ├── /reflections
      └── /settings
```

Page-by-page specification

`/login` and `/register`

Layout: Centred card, no sidebar, dark/light mode toggle in corner.

Login form fields:

- Email (type=email, required)
- Password (type=password, required)
- "Remember this device" checkbox — stores device label for refresh token
- Submit button — shows loading spinner while request in flight

Behaviour:

1. Form submits to `POST /api/v1/auth/login`
2. On success: store `accessToken` in `AuthProvider` memory, redirect to `/dashboard`
3. On 401: show "Invalid email or password"
4. On 429: show "Too many attempts. Try again in X minutes."

Register form fields: Name, Email, Password, Confirm Password.

`/dashboard`

Type: SSR page — data fetched server-side using the session cookie before render.

Layout: Three-column on desktop, stacked on mobile.

Sections:

Section	Content
Streak card	Current streak count, longest streak, last log date
Recent logs	Last 5 log entries with mood indicator and tags
Goal progress	In-progress goals with milestone completion percentage
Top skills	5 skills with highest log activity this month
Weekly prompt	CTA to write this week's reflection (if not yet submitted)

`/logs`

Type: Client component with infinite scroll.

List view: Each log card shows:

- Title, first 2 lines of body (markdown rendered)
- Mood emoji, tags, linked skills as badges
- Relative timestamp ("2 days ago")
- Source badge — MANUAL / GITHUB PUSH / GITHUB PR
- Edit and Delete actions (owner only)

Filters (sidebar panel):

- Date range picker
- Tag multi-select (shows all tags user has used)
- Skill filter
- Mood filter
- Source filter

Infinite scroll: Fetches next page using `nextCursor` when user scrolls to 80% of page. Uses `IntersectionObserver`.

New log button: Opens slide-over panel with:

- Title input
- Markdown editor with preview toggle

- Mood selector (5 emoji options)
- Tag input with autocomplete from existing tags
- Skill multi-select (from user's skill list)
- Goal multi-select (from in-progress goals)

`/goals`

List view: Card grid. Each card shows:

- Goal title, status badge (IN_PROGRESS / COMPLETED / PAUSED)
- Progress bar: completed milestones / total milestones
- Target date with overdue indicator
- Days since last log linked to this goal

Goal detail page `/goals/[id]`:

- Full goal description
- Milestone checklist — click to complete, drag to reorder
- Timeline: logs linked to this goal in chronological order
- Edit title / description / target date

`/skills`

Layout: Two sections — radar chart (top), skill table (bottom).

Radar chart: Shows top 8 skills plotted on a radar/spider chart. Built with Recharts `RadarChart`. Data fetched from `/api/v1/skills/me/graph` (Redis cached).

Skill table columns: Skill name, Category, Level (1–10 with visual bar), Log count, Last logged date.

Skill detail: Click a skill → slide-over showing all logs that tagged that skill, level progression over time as a line chart.

`/reflections`

Weekly reflection form:

Four text areas (markdown enabled):

1. **Wins** — what went well this week

2. **Struggles** — what was hard or didn't go as planned
3. **Learning** — the most important thing you learned
4. **Next week** — what you intend to focus on

Plus: mood selector for the week overall.

History: Accordion list of past reflections by week. Click to expand and read. Cannot edit past reflections (append-only by design — your record of how you actually felt that week).

`/settings`

Tabs: Profile · Security · Webhooks · Integrations · Share

Profile tab: Name, email, timezone (used for weekly cron timing).

Security tab:

- Change password
- Active sessions list — shows device label, last seen, location
- "Log out all other devices" button

Webhooks tab:

- List of configured outbound webhooks (URL, events, last triggered, status)
- "Add webhook" form: URL, select events, generates and displays HMAC secret
- Delivery log: last 20 deliveries per webhook with status code and response time

Integrations tab:

- GitHub integration setup:
  1. Click "Connect GitHub"
  2. Page shows unique inbound webhook URL and auto-generated secret
  3. Copy-paste these into GitHub repo Settings → Webhooks
  4. Test webhook button — sends a ping and confirms receipt

Share tab:

- Toggle to enable/disable mentor share link
- Shows shareable URL: `https://yourdevlog.app/mentor/{shareToken}`
- "Regenerate link" — invalidates old token, issues new one
- Mentor view is read-only: sees logs, goals, skills. Cannot see reflections (private by default), settings, or auth data.

`/mentor/[token]` (public)

Auth: No login required. Validates `shareToken` against DB. 404 if not found or sharing disabled.

Shows: Dashboard summary, log feed (read-only), goal list, skill radar chart.

Does not show: Reflections, settings, security info, webhook config.

Component inventory

Component	Type	Responsibility
`AuthProvider`	Client	Holds access token in memory. Runs silent refresh on mount and before expiry. Provides `useAuth()` hook.
`RoleGate`	Client	Wraps UI sections that require a specific role. Reads role from auth context. Renders children or null.
`LogCard`	Server / Client	Renders a single log entry. Handles edit/delete actions.
`MarkdownEditor`	Client	Textarea with live markdown preview toggle.
`SkillBadge`	Server	Pill badge showing skill name and level. Colour-coded by category.
`GoalProgress`	Client	Progress bar derived from milestone completion count.
`InfiniteScroll`	Client	Wraps any list. Uses `IntersectionObserver` to fetch next page.
`SlideOver`	Client	Slide-in panel from the right for create/edit forms.
`RadarChart`	Client	Recharts `RadarChart` for skill graph. Receives pre-computed data from `/skills/me/graph`.

`AuthProvider` — token refresh logic

```typescript
```

```
// apps/web/components/AuthProvider.tsx
'use client'

export const AuthProvider = ({ children }) => {
  const [accessToken, setAccessToken] = useState<string | null>(null)

  // On mount: try to get a new access token using the HttpOnly refresh cookie
  useEffect(() => {
    silentRefresh()
  }, [])

  const silentRefresh = async () => {
    // The browser automatically sends the HttpOnly cookie with this request
    const res = await fetch('/api/v1/auth/refresh', { method: 'POST', credentials: '
    if (res.ok) {
      const { accessToken } = await res.json()
      setAccessToken(accessToken)
      // Schedule next refresh 1 minute before expiry (access token TTL = 15 min)
      setTimeout(silentRefresh, 14 * 60 * 1000)
    } else {
      // Refresh token expired or revoked — user must log in again
      setAccessToken(null)
    }
  }

  return <AuthContext.Provider value={{ accessToken, silentRefresh }}>{children}</Au
}
```

---

15. Build Phases & Implementation Order

Build in this order. Each phase produces working, shippable software — never a half-built system.

Phase 1 — Foundation (Week 1-2)

Goal: Running monorepo, database, and a single working API endpoint.

1. Initialise Turborepo monorepo with `apps/web`, `apps/api`, `packages/db`, `packages/config`
2. Set up TypeScript strict mode in all packages
3. Write `packages/config/index.ts` — load and Zod-validate all env vars. App should crash on startup if any required var is missing

4. Write `packages/db/schema.prisma` — all models from the schema section above
5. Run first migration: `prisma migrate dev --name init`
6. Write `packages/db/seed.ts` — create Owner user, seed skill catalog
7. Set up Express app in `apps/api/src/app.ts` — helmet, cors, morgan, JSON body parser
8. Write `apps/api/src/shared/errors.ts` — AppError, NotFoundError, UnauthorizedError, ForbiddenError, ValidationError
9. Write global error handler middleware — catches all thrown errors, returns correct status + JSON envelope
10. Write `apps/api/src/shared/logger.ts` — Pino with request ID and user ID fields
11. Write health check endpoint `GET /health` — returns `{ status: 'ok', db: 'ok', redis: 'ok' }`
12. `docker-compose.yml` with Postgres and Redis services
13. Verify: `docker compose up`, `npm run dev`, hit `GET /health` → 200

Done when: Monorepo builds cleanly, DB migrations run, health check returns 200.

Phase 2 — Authentication (Week 3–4)

Goal: Full JWT auth flow working end to end.

1. Generate RSA key pair for JWT signing: `openssl genrsa -out private.pem 2048` and extract public key
2. Write `auth.service.ts` — `issueTokens()`, `verifyAccessToken()`, `rotateRefreshToken()`, `detectReuse()`
3. Write `auth.middleware.ts` — `verifyAccessToken`, `attachUser`
4. Write `auth.controller.ts` + `auth.routes.ts` — all 6 endpoints from the spec
5. Write `session.ts` — Redis-backed encrypted session for admin flows
6. Write `rbac/permissions.ts` and `rbac/rbac.middleware.ts`
7. Set up Next.js `AuthProvider.tsx` — silent refresh on mount, token in memory
8. Build `/login` and `/register` pages with forms
9. Build `apps/web/lib/api.ts` — fetch wrapper that injects Authorization header and handles 401 → trigger silent refresh → retry

Done when: Register → login → access a protected route → token expires → silent refresh → still works → logout → protected route returns 401.

Phase 3 — Core CRUD (Week 5–7)

Goal: Logs, Goals, and Skills modules fully working.

1. Write `logs.repository.ts` — implement cursor-based pagination
2. Write `logs.service.ts` — create/update/delete with skill side effects in a transaction
3. Write `logs.controller.ts` + `logs.routes.ts`
4. Write `goals.repository.ts`, `goals.service.ts`, `goals.controller.ts`, `goals.routes.ts`
5. Write `skills.repository.ts`, `skills.service.ts` — with cache-aside using Redis
6. Write `skills.controller.ts` + `skills.routes.ts`
7. Build frontend: `/logs` page with infinite scroll list and create slide-over
8. Build frontend: `/goals` page with milestone checklist
9. Build frontend: `/skills` page with radar chart

Done when: Can create a log entry, link it to a goal and a skill, see skill level update, see goal milestone progress reflect in UI.

Phase 4 — Webhooks (Week 8–9)

Goal: GitHub push creates a log entry automatically.

1. Write `webhook.service.ts` — HMAC verification, idempotency check via Redis
2. Write `webhook.routes.ts` — `POST /api/v1/webhooks/github`
3. Set up BullMQ in `shared/queue.ts` — producer and worker scaffolding
4. Write `webhook.dispatcher.ts` — outbound dispatch with BullMQ and exponential backoff retry
5. Write webhook CRUD endpoints (register, list, delete)
6. Build frontend: `/settings/webhooks` — list, add, delivery history
7. Build frontend: `/settings` integrations tab — GitHub setup instructions with copy-paste URL and secret
8. Test: configure a real GitHub webhook pointing to `localhost` via ngrok, push a commit, verify log entry is auto-created

Done when: A GitHub commit push creates a log entry. An outbound webhook fires and delivers to a test URL. A failed delivery retries automatically.

Phase 5 — Workflows & Reflections (Week 10–11)

Goal: Automation rules fire. Weekly reflections work.

1. Write `workflow.engine.ts` — `emit()` method, BullMQ action worker
2. Wire `workflowEngine.emit()` calls into goals.service and logs.service (after transactions commit)
3. Write `workflow.cron.ts` — weekly reflection reminder, daily streak check
4. Write `reflections` module — CRUD endpoints (create is upsert by weekStart)
5. Build frontend: `/reflections` — weekly form and history accordion
6. Build frontend: workflow rule creation UI in `/settings`

Done when: Complete a goal → outbound webhook fires if configured. Miss 3 days → streak event fires. Weekly reflection reminder cron runs and triggers configured actions.

Phase 6 — Performance, Sharing & Polish (Week 12)

Goal: Production-ready hardening.

1. Audit every DB query — add missing indexes, check for N+1 queries using Prisma's query log
2. Add rate limiting on all auth endpoints — express-rate-limit backed by Redis
3. Implement share token system — generate, validate, render `/mentor/[token]` page
4. Write graceful shutdown in `server.ts`
5. Add request logging with request ID that traces across API logs
6. Write `README.md` documenting local setup
7. Final end-to-end test of all flows

Done when: All six phases green. Share link works. No N+1 queries. Auth endpoints rate-limited. App shuts down cleanly on SIGTERM.

---

16. Environment Variables

```bash
```

```
# packages/config/.env.example

# ── Database ──────────────────────────────────────────
DATABASE_URL="postgresql://devlog:devlog@localhost:5432/devlog"

# ── Redis ─────────────────────────────────────────────
REDIS_URL="redis://localhost:6379"

# ── JWT ───────────────────────────────────────────────
JWT_PRIVATE_KEY_PATH="./keys/private.pem"       # RS256 private key for signing
JWT_PUBLIC_KEY_PATH="./keys/public.pem"          # RS256 public key for verifying
JWT_ACCESS_TOKEN_TTL="900"                       # seconds — 15 minutes
JWT_REFRESH_TOKEN_TTL_DAYS="7"

# ── Session ───────────────────────────────────────────
SESSION_SECRET="min-32-char-random-string"       # AES-256 encryption key for sessio
SESSION_TTL_SECONDS="7200"                       # 2 hours

# ── App ───────────────────────────────────────────────
PORT="4000"
NODE_ENV="development"
API_BASE_URL="http://localhost:4000"
WEB_BASE_URL="http://localhost:3000"
CORS_ORIGIN="http://localhost:3000"

# ── GitHub Webhook ────────────────────────────────────
GITHUB_WEBHOOK_SECRET="random-string-you-also-paste-into-github"

# ── Email (for workflow notifications) ────────────────
SMTP_HOST="smtp.example.com"
SMTP_PORT="587"
SMTP_USER="noreply@example.com"
SMTP_PASS="your-smtp-password"
EMAIL_FROM="DevLog <noreply@example.com>"
```

Startup validation (`packages/config/index.ts`):

```typescript
```

```
import { z } from 'zod'

const configSchema = z.object({
  DATABASE_URL:           z.string().url(),
  REDIS_URL:              z.string().url(),
  JWT_PRIVATE_KEY_PATH:   z.string(),
  JWT_PUBLIC_KEY_PATH:    z.string(),
  SESSION_SECRET:         z.string().min(32),
  PORT:                   z.coerce.number().default(4000),
  NODE_ENV:               z.enum(['development', 'test', 'production']).default('deve
})

const parsed = configSchema.safeParse(process.env)
if (!parsed.success) {
  console.error('Invalid environment config:', parsed.error.format())
  process.exit(1)   // ← fail fast — never start with broken config
}

export const config = parsed.data
```

This PRD is the single source of truth for the DevLog project. Every module, endpoint, schema field, UI component, and architectural decision is specified here. Start with Phase 1 and build forward — the order matters.